Developing Apps on QT

In this part of the series, we deal with file operations in Qt.

Part—6

Most of the time, we hear that every thing in UNIX is a file, as in Linux. As per my knowledge, except for the network drivers, every thing is a file in Linux. We store data, software settings, etc, in files, so for application programmers, the basics of file operations are important. Here, we're using C++ with Qt.

QFile—a robust file interface

Qt provides a robust interface to files; file operations are easy to perform. Look at our sample code, *main.cpp:*

```
#include <QtCore/QCoreApplication>
#include <QtCore>
int main(int argc, char *argv[]) {
    QCoreApplication a(argc, argv);
    QFile file;
    // Set the file name
    file.setFileName("./sample");
    // Open the file
    if(!file.open(QIODevice::WriteOnly | QIODevice::Text)) {
        qDebug() << "File opening failed";
        return -1;
    }
    qDebug() << "File opened";
    // Write data to file
    file.write("Welcome to Open World");
    // Don't forget to close the file
    file.close();
    return a.exec();
}
```

The *QFile* class provides an interface for file read-write operations. *QFile* inherits from *QIODevice*, which provides the interface for I/O to and from devices. We won't go into the details now, but we will concentrate on *QFile*. We created a *QFile* object, and then specified the path and name of the file by calling the *setFileName()* member function. Next, we open the file, which can be done in one of three modes: read, write or read-write, which needs to be specified while opening the file. We used (*QIODevice::WriteOnly* | *QIODevice::Text*) (the enums of the *QIODevice* class) to specify the mode. If required, we can do a bitwise OR of the options, like above, where we opened the file with *Text* mode so that while reading and writing, line terminators are translated to local encoding.

Writing and reading is simple with the *write()* and *read()* member functions. Never forget to close the file when done—it tells the OS to flush the data from RAM to the hard disk.

Working with streams

A more convenient way of accessing text files is to use the *QTextStream* class for reading and writing text. The target can be a file, string or a byte array. Let's look at the sample code:

```
QFile file;
    file.setFileName("./sample");
    if(!file.open(QIODevice::ReadWrite | QIODevice::Text)) {
        qDebug() << "File opening failed";
        return -1;
    }
    QTextStream stream(&file);
    stream << "Knowledge-gives-Happiness";
```